T3. Modelado Estructural

Las clases son los bloques de construcción más importantes del paradigma orientado a objetos. Una **clase** es una descripción de un conjunto de **objetos** que comparten **atributos**, **operaciones** (llamados métodos en JAVA), **relaciones** y **semántica**. Son **abstracciones** independientes del lenguaje de programación capaces de implementar una o más **interfaces**.

Una clase se especifica mediante un **nombre** único dentro del paquete, una serie de **atributos** (propiedades compartidas por todos los representantes de la clase que determinan el su estado mediante una expresión nominal, siguen al siguiente sintaxis: **"visibilidad nombre : tipo = valor defecto"**) y una serie de operaciones (servicios que pueden ser requeridos a cualquier instancia y definen el comportamiento mediante una expresión verbal, siguen la siguiente sintaxis: **"visibilidad nombre (parámetros) : retorno"**).

Los objetos de una misma clase tienen los mismos tipos de **estado** y **comportamiento**. A nivel abstracto, los atributos y las operaciones son un medio para asumir responsabilidades (contratos para una clase). Una clase bien estructurada tiene pocas responsabilidades y la mayoría de las clases colaboran entre sí para ejercerlas.

Tarjetas CRC

Las tarjetas **CRC** (Clase-Responsabilidad-Colaborador) no forman parte de UML y cada una consta de: el **nombre** de la clase, las **responsabilidades** (frases cortas que resumen lo que deberían hacer sus instancias) y **colaboraciones** (clases en las que se apoyan para asumir dichas responsabilidades). Suponen una serie de **ventajas**, ya que fomentan discusión entre desarrolladores, permiten concebir interacciones entre objetos sin necesidad de diagramas, previenen las clases limitadas a *contenedores de datos*. Pero suponen un mayor **riesgo** ya que generan largas listas de responsabilidades de bajo nivel.

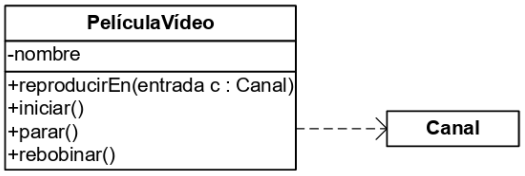
Pedido	
Responsabilidades	**Colaboradores**
Comprobar artículos en stock	LineaPedido
Determinar precio	Cliente
Comprobar validez de pago	
Enviar a dirección de reparto	

Modelado de clases

Para el **modelado de clases** se debe identificar y definir las abstracciones que se utilizan para describir el **dominio** y repartir **adecuadamente** las responsabilidades entre las clases. Además de proporcionar a cada clase los atributos, operaciones y relaciones necesarias para asumir sus responsabilidades. Para cada clase se debe distinguir la interfaz de la implementación.

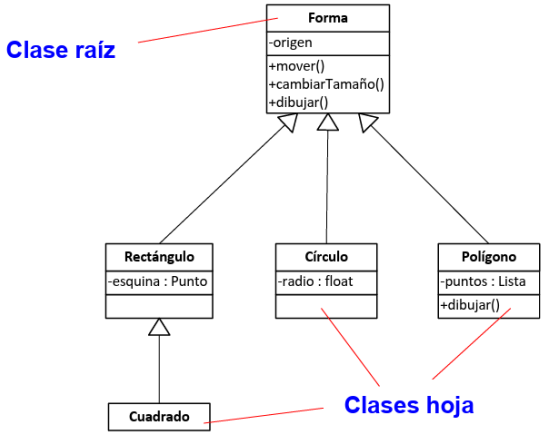
La mayoría de las clases no están aisladas, sino que **colaboran**. Existen 3 tipos de **relaciones** que se deben establecer de forma equilibrada:

- **Dependencia**: son relaciones de uso. Declara que los cambios en la planificación de un elemento pueden afectar a otro que lo usa, se emplea para indicar **utilización**. Es la relación más débil de las tres.



- **Generalización:** relación entre un elemento general (padre) y otro específico (hijo), de modo que corresponde a "es un tipo de...". Se rige por el principio de **sustitución**, se deben evitar ciclos y equilibrar las jerarquías. Las subclases **heredan propiedades** de las superclases; se pueden añadir otras clases y redefinir operaciones heredadas (polimorfismo).

  Hay que tener en cuenta que en UML se admite la **herencia múltiple**, al contrario que en JAVA. Existen dos tipos de clases: una clase **raíz** que no tienen padres (de la que todos heredan) y una o varias clases **hoja** que no tienen hijos.

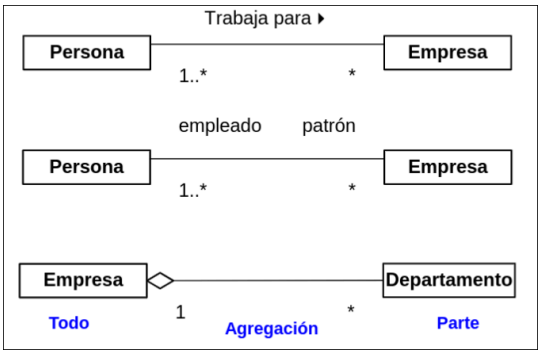


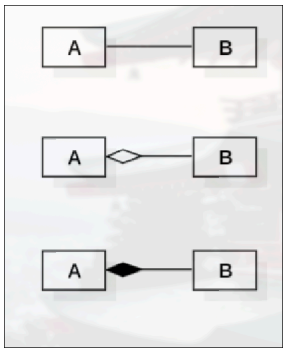
  Para el **modelado de herencia simple** se identifican las responsabilidades, atributos y operaciones comunes a varias clases. Posteriormente, se elevan estas propiedades a una clase más **general** (se crea si no existe) y se construye una jerarquía especificando las generalizaciones necesarias.

- **Asociación:** es una relación estructural entre objetos. Se usa para vincular clases del **mismo nivel** (al contrario que la dependencia y la generalización). Además, existen **adornos**:

  - **Nombre**: etiqueta que describe la relación.
  - **Rol**: papel que juega cada clase relacionada.

- **Multiplicidad**: número de instancias participantes.
- **Agregación**: es como entidad débil-fuerte en BD (asociación todo-partes) pero las partes pueden existir sin el todo.
- **Composición**: los objetos parte no pueden existir sin el todo.





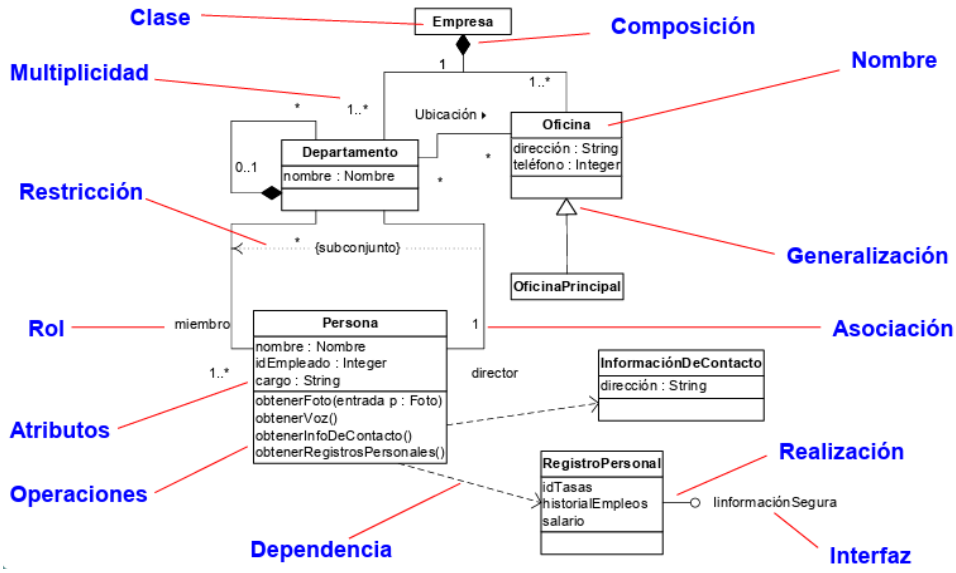
Solo existen relaciones binarias en UML y los nombres de las asociaciones son únicos.

Nota: El triángulo de la primera asociación únicamente ayuda a leer el diagrama. Nunca se indica el nombre de la asociación y los roles de la misma, o se hace uno o el otro pero nunca ambos.

Nota: A la hora de trazar las relaciones se debe usar consistentemente líneas rectas y oblicuas, se evitarán cruces de líneas y se deberán mostrar sólo las relaciones necesarias.

Diagrama de clases

El **diagrama de clases** es la columna vertebral del modelado orientado a objetos, con clases, interfaces, colaboraciones y relaciones (dependencia, generalización, asociación y realización). Estos diagramas modelan la **vista de diseño** y son la base para diagramas de **paquetes**, **componentes** y **despliegue**.



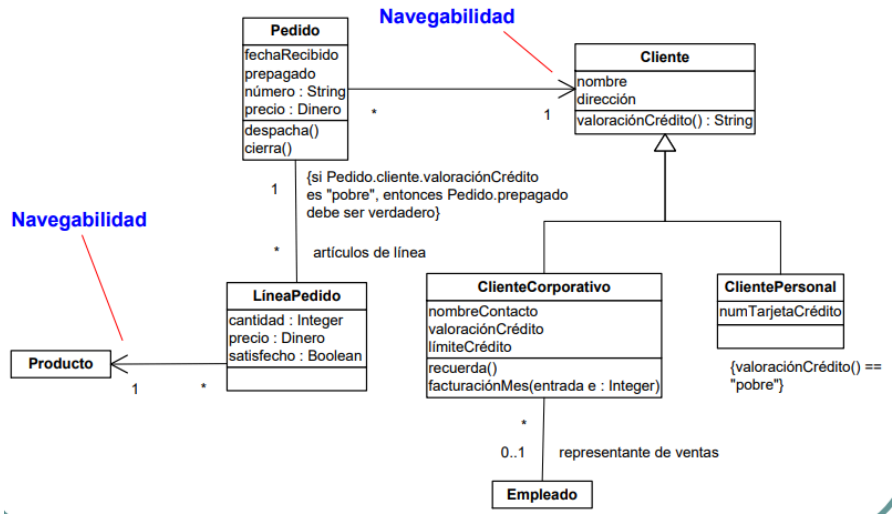
Navegatividad

La navegatividad en un diagrama de clases de UML indica la **dirección** en la que una **clase** puede **acceder a otra dentro de una asociación**. Es decir, define que clase "conoce" a la otra y puede utilizar sus atributos o métodos.

La navegatividad **solo** se aplica a **asociaciones** entre clases. La asociación es la relación estructural entre clases, mientras que la navegatividad es la dirección de esa relación. Tipos:

- **Unidireccional**: Solo una clase conoce a la otra.
  $A \rightarrow B$, A conoce a B y puede invocar sus métodos.

- **Bidireccional**: Ambas clases coñécense mutuamente. $A \leftrightarrow B \quad o \quad A - B$.



Modelado con Diagramas de clases

En UML, los diagramas de clases se utilizan para **representar la estructura estática de un sistema**, mostrando clases, relaciones y organización del dominio. El primer paso del modelado es el **vocabulario de un dominio,** que consiste en identificar conceptos clave del dominio y las relaciones entre ellos. El segundo paso es la **estructura de colaboraciones**, una colaboración es un conjunto de elementos que proporcionan un comportamiento colaborativo para que el resultado sea **mayor que la suma de las partes**. La tercera parte es el **esquema conceptual de la base de datos**, el diagrama de clases puede usarse para diseñar BD relacionales, BD orientadas a objetos o Sistemas híbridos, **modelando** tablas (clases), relaciones (asociaciones) y atributos (columnas).

Ingeniería directa e inversa

- **Ingeniería directa:** Es el proceso de pasar los modelos de UML a **código** aplicando reglas de correspondencia. Para ello, UML no especifica correspondencia con ningún lenguaje de programación y produce mucha pérdida de información. Se necesita **identificar** las **reglas de correspondencia**, restringir ciertas **características** de UML, especificar el **lenguaje** de programación y usar las **herramientas** apropiadas.

- **Ingeniería inversa:** Es el proceso contrario, del código al **modelo**. Produce mucha cantidad de información pero no permite recrear exactamente el modelo.

Nota: No se debe usar notación compleja , comenzar por elementos básicos y no modelar absolutamente todo. Se debe ajustar la perspectiva de los modelos a la etapa del proyecto: **modelos conceptuales** (análisis, representan el problema y no incluyen detalles técnicos), **modelos de especificación** (diseño, son modelos más detallados y preparan la implementación) y **modelos de implementación** (son cercanos al código e incluyen detalles técnicos).

Notas

Las **notas** son el tipo de *adorno* aislado más importante en UML. Permiten mostrar **comentarios** o **restricciones** en formato libre; añaden al modelo requisitos, observaciones, revisiones y explicaciones. Hay que tener en cuenta que se debe enlazar la nota al elemento correspondiente y no es necesario que sea visible en todos los lugares donde aparezca el elemento o la relación (vale con aparecer una sola vez). Sirve para mover comentarios largos o complejos a un solo texto y **sólo** mantener las notas **significativas** no deducibles del propio modelo.